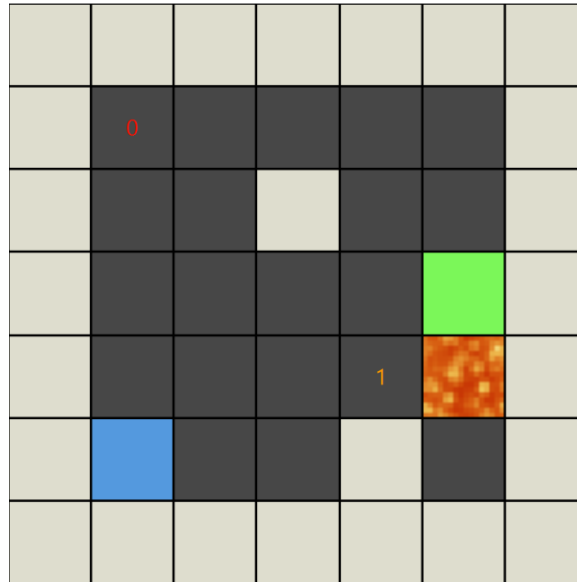


LAPORAN TUCIL_3 STIMA

I Gusti Ngurah Alit Dharma Yudha | 13524072 | 13524072@std.stei.itb.ac.id

I. Latar Belakang

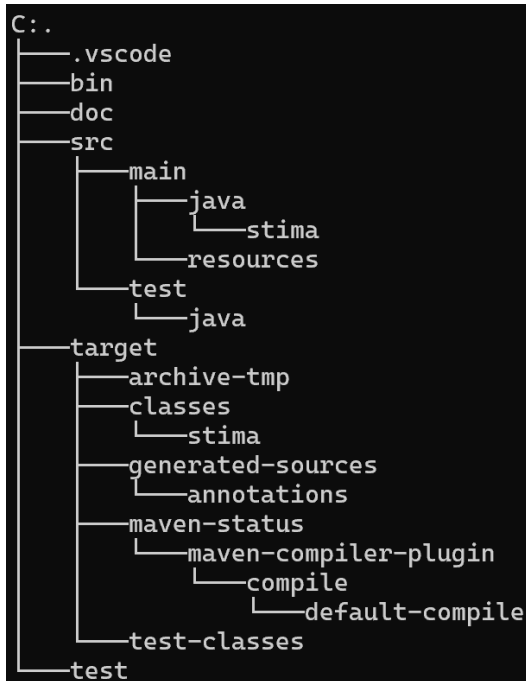


Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Jika dalam Map terdapat *checkpoint* (angka), karakter *player* harus melewati semua *checkpoint* dalam urutan numerikal sebelum mengunjungi petak akhir. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih). Dalam tugas kecil ini kami ditugaskan untuk membuat program yang dapat menghasilkan Solusi dari permainan ini menggunakan algoritma UCS (Uniform Cost Search), GBFS (Greedy-Best First Search), dan A*.

II. Implementasi & Algoritma

Secara kasar program yang saya buat memiliki struktur file seperti berikut,

```
main
├── java
│   └── stima
│       ├── App.java
│       ├── controller.java
│       ├── Launcher.java
│       ├── matrix.java
│       └── the_i.java
└── resources
    ├── lava.png
    ├── main.fxml
    └── spiki.png
```



/bin berisi file .jar, /doc berisi laporan ini, /src berisi source code dari program, /target hanya untuk hasil build, ./test berisi test-case yang akan digunakan. Program utama *App.java* terletak di src/main/java/stima,

```
public class App extends Application{

    @Override
    public void start(Stage stage) throws Exception {
        try{
            FXMLLoader loader = new FXMLLoader(getClass().getResource("/main.fxml"));
            Image spiki = new Image(getClass().getResourceAsStream("/spiki.png"));
            Parent root = loader.load();
            Controller controll = loader.getController();
            Scene scene = new Scene(root,Color.DIMGRAY);

            scene.setOnKeyPressed(new EventHandler<KeyEvent>() {

                @Override
                public void handle(KeyEvent event){
                    controll.handle_buttons(event);
                }
            });

            stage.getIcons().add(spiki);
            stage.setTitle(value: "ice sliding puzzle solver");
            stage.setScene(scene);
            stage.setResizable(value: false);
            stage.show();
        } catch (Exception e){
            e.printStackTrace();
        }
    }

    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception{
        launch(args);
    }
}
```

App.java sendiri hanya melakukan inisialisasi stage/window untuk GUI, kemudian ada *controller.java* yang mengatur semua fungsionalitas dari setiap komponen GUI. Kemudian class-class sisa-nya yakni, *matrix.java*, *the_i.java*, dan beberapa subclass di *matrix.java* karena saya terlalu malas untuk membuatnya menjadi file sendiri, digunakan untuk melakukan komputasi dan sebagai data structure saja. Terakhir *Launcher.java* untuk melakukan launch seluruh program saat dijadikan file java executable. Ini diperlukan karena program tidak bisa langsung di jalankan dari kelas yang merupakan turunan dari Application class (*App.java*) jika dalam bentuk *jar*.

Dalam program ini ada beberapa metode pencarian yang bisa dilakukan, UCS (Uniform Search Cost), GBFS (Greedy-Best First Search), dan A*. Seluruh metodenya diimplementasi menggunakan priority queue sebagai berikut,

```
public result search(int mode){ // mode: algorithm
    HashSet<String> set = new HashSet<>(); // so we dont overlap/re-do paths
    PriorityQueue<State> que = new PriorityQueue<>();
    ArrayList<State> iter = new ArrayList<>();
    // ucs
    if(mode==0){
        que = new PriorityQueue<>((a,b) -> Double.compare(a.cost, b.cost));
    }
    //gbfs
    else if(mode==1){
        que = new PriorityQueue<>((a,b) -> Double.compare(calculate_line_distance(a.pos.r, a.pos.c), calculate_line_distance(b.pos.r, b.pos.c)));
    }
    // a*
    else if(mode==2){
        que = new PriorityQueue<>((a,b) -> Double.compare(calculate_line_distance(a.pos.r, a.pos.c)+a.cost, calculate_line_distance(b.pos.r, b.pos.c)+b.cost));
    }

    State start = new State(new position(this.player.r, this.player.c), new ArrayList<>(), new ArrayList<>(), cost: 0);
    que.add(start);
    while(!que.isEmpty()){
        State current = que.poll();
        iter.add(current);
        if(set.contains(current.getKey())) continue;
        if(current.pos.r==this.goal.r && current.pos.c==this.goal.c && current.visited.size()==number){
            return new result(iter, current.path, current.cost);
        }
        set.add(current.getKey());

        for(direction dir : direction.values()){
            ArrayList<Integer> copy_visited = new ArrayList<>(current.visited);
            simulate_move_result result = simulate_move(dir, current.pos, copy_visited);
            if(result==null) continue;

            ArrayList<direction> new_path = new ArrayList<>(current.path);
            new_path.add(dir);
            State next_state = new State(result.end_pos, new_path, copy_visited, current.cost+result.cost);

            if(!set.contains(next_state.getKey())){
                que.add(next_state);
            }
        }
    }

    return new result(iter, path: null, cost: 0); // not found
}
```

HashSet digunakan agar tidak ada pengulangan *Path* yang sudah diproses, Priority Queue digunakan untuk menentukan node selanjutnya yang akan diproses.

Perbedaan dari setiap algoritma hanya pada pengurutan Priority Queuenya, untuk UCS menggunakan $g(h)$ yakni cost tempuh, GBFS menggunakan $f(n) = h(n)$ Dimana heuristiknya berupa jarak Euclidean antar petak, dan A* menggunakan penjumlahan dari keduanya jadi $f(n) = g(n) + h(n)$. Karena itu, A* lebih seimbang, tidak serakus GBFS dan biasanya lebih terarah daripada UCS. Secara teori, UCS paling aman untuk optimalitas, GBFS paling cepat tetapi bisa malah memilih jalur yang mahal, A* berusaha mendapatkan Solusi optimal dengan jalur yang efisien.

Heuristik yang dipilih untuk A* umumnya admissible jika setiap petak yang dilewati tidak memiliki cost yang kurang dari sama dengan 1. Tetapi jika terjadi kasus dimana terdapat satu gerakan yang bisa membawa *player* sangat dekat atau langsung ke *goal* dengan cost yang lebih kecil dari *Euclidean distance*, maka $h(n)$ bisa melebihi-lebihkan *cost* sebenarnya, jadi A* kehilangan optimalitas-nya.

Berikut merupakan contoh alur program, misal kita punya Map sebesar 5x5 sebagai berikut,

X Cost: 999	X Cost: 999	X Cost: 999	X Cost: 999	X Cost: 999
X Cost: 999	Z Cost: 1	* Cost: 1	* Cost: 1	* Cost: 1
X Cost: 999	* Cost: 1	* Cost: 1	* Cost: 1	X Cost: 999
X Cost: 999	X Cost: 999	X Cost: 999	O Cost: 1	X Cost: 999
X Cost: 999	X Cost: 999	X Cost: 999	X Cost: 999	X Cost: 999

X merepresentasikan tembok, Z merepresentasikan player, O merepresentasikan titik akhir, dan * merepresentasikan petak biasa. Proses pencariannya sebagai berikut (misal petak paling atas kiri merupakan (0,0)),

Current Node	Node Hidup
(1,1) cost: 0	D(2,1) cost: 1
D(2,1) cost: 1	DR(2,3) cost: 3,
DR(2,3) cost: 3	DRU(1,3) cost: 4, DRD(3,3) cost: 4
DRU(1,3) cost: 4	DRD(3,3) cost: 4
DRD(3,3) cost: 4	GOALLL

(UCS)

Current Node	Node Hidup
(1,1) h: 2.828	D(2,1) h: 2.236
D(2,1) h: 2.236	DR(2,3) h: 1,
DR(2,3) h: 1	DRU(1,3) h: 2, DRD(3,3) h: 0
DRD(3,3) h: 0	GOALLL

(GBFS)

Current Node	Node Hidup
(1,1) f: $0 + 2.828 = 2.828$	D(2,1) f: $1 + 2.236 = 3.236$
D(2,1) f: 3.236	DR(2,3) f: $3 + 1 = 4$,
DR(2,3) f: 4	DRU(1,3) f: $4 + 2 = 6$, DRD(3,3) f: $4 + 0 = 4$
DRD(3,3) cost: 4	Goal

(A*)

Hasil dari pencarian kemudian akan disimpan dalam struct data seperti berikut. *Path* berupa jalur solusi, *iterations* merupakan setiap kondisi board pada saat dilakukan *processing* iteration dalam bentuk data struct *State* yang ada dibawah, dan *cost* berupa cost akhir dari hasil pencarian. Dalam struct *State*, *visited* menyimpan checkpoint mana saja yang sudah dikunjungi, uhhhhh melihatnya lagi sekarang, kayaknya itu bisa dalam bentuk *int* saja tidak usah List.... Oh well.

```
public class result{
    public ArrayList<direction> path;
    public ArrayList<State> iterations;
    public double cost;

    public result(ArrayList<State> iter, ArrayList<direction> path, double cost){
        this.iterations=iter;
        this.path=path;
        this.cost=cost;
    }
}
```

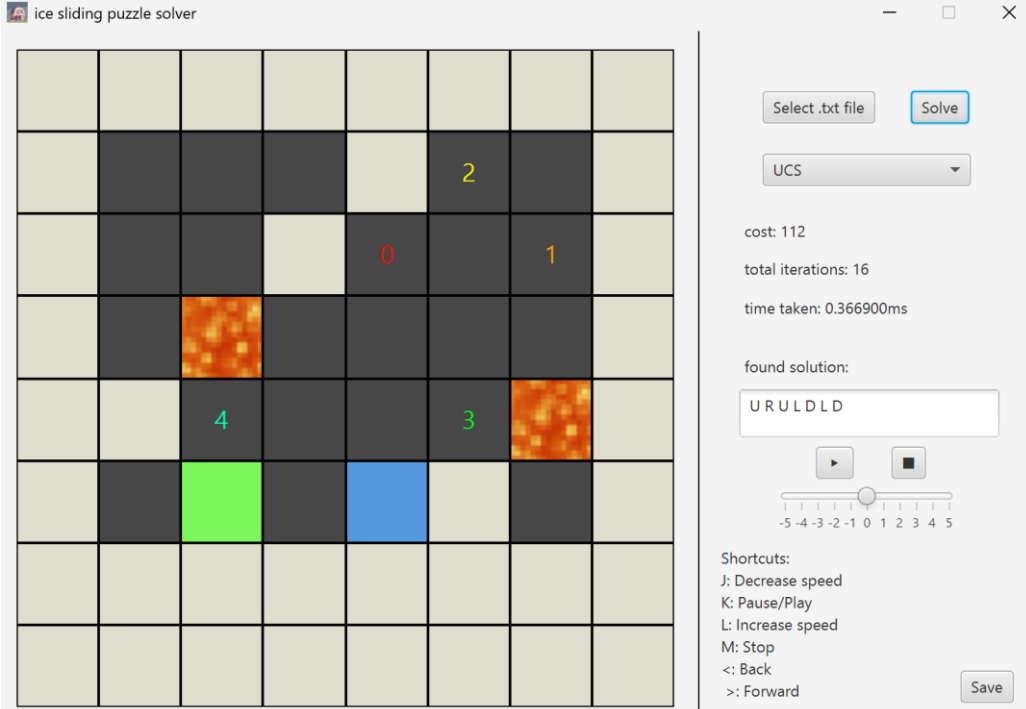
```
public class State{
    public position pos;
    public ArrayList<direction> path;
    public ArrayList<Integer> visited;
    public double cost;

    public State(position pos, ArrayList<direction> path, ArrayList<Integer> visited, double cost){
        this.pos=pos;
        this.path=path;
        this.visited=visited;
        this.cost=cost;
    }
    public String getKey(){
        return pos.r + "," + pos.c + "," + visited.toString();
    }
}
```

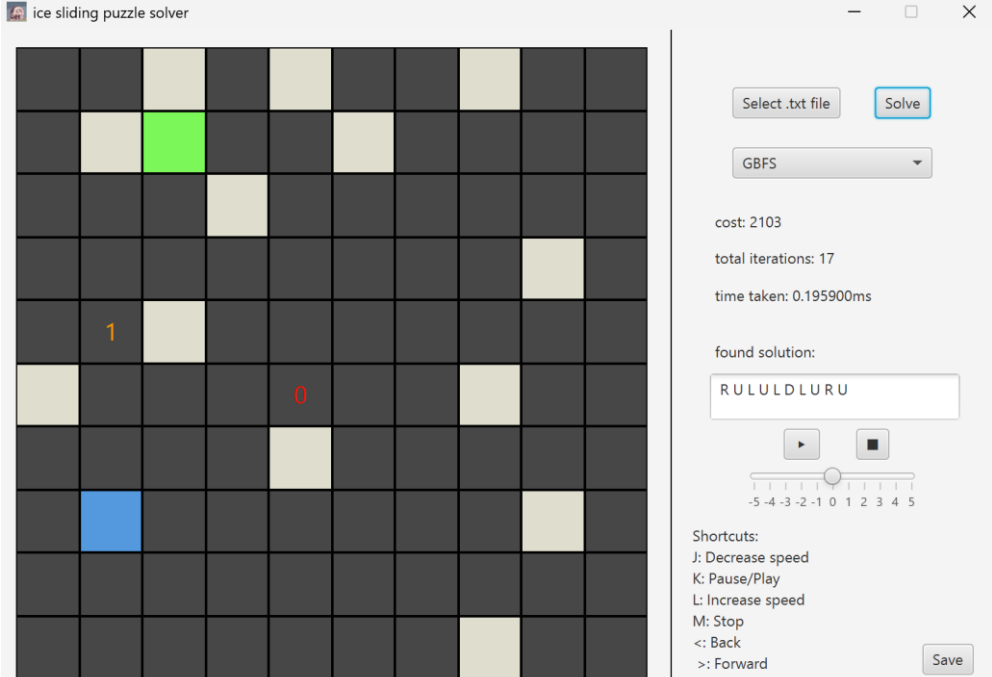
result tersebut kemudian diproses di *controller.java* untuk ditampilkan di GUI-nya.

III. Pengujian

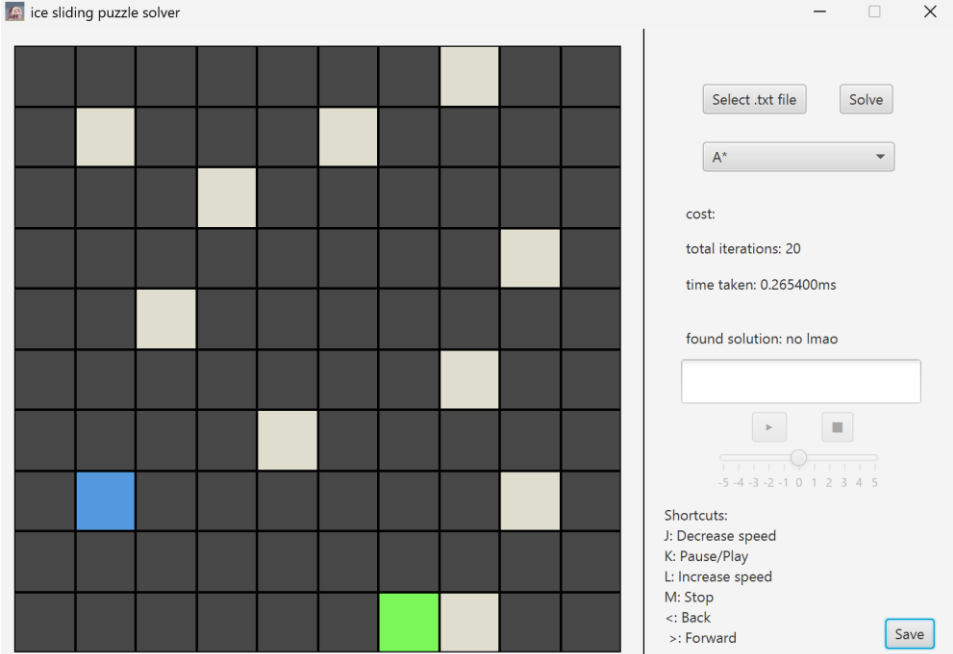
1. Test case 1

Input	<pre> 8 8 XXXXXXXXX X***X2*X X**X0*1X X*L****X XX4**3LX X*O*ZX*X XXXXXXXXX XXXXXXXXX 999 999 999 999 999 999 999 999 999 8 8 8 999 8 8 999 999 8 8 999 8 8 8 999 999 8 7 8 8 8 8 999 999 999 8 8 8 8 7 999 999 8 8 8 8 999 8 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 </pre> Metode: UCS
Output	
Analisis	Program menemukan solusi optimal yang melewati semua checkpoint dan tidak melewati lava. Setelah pengujian lebih lanjut, semua metode menghasilkan hasil cost dan solusi yang sama.

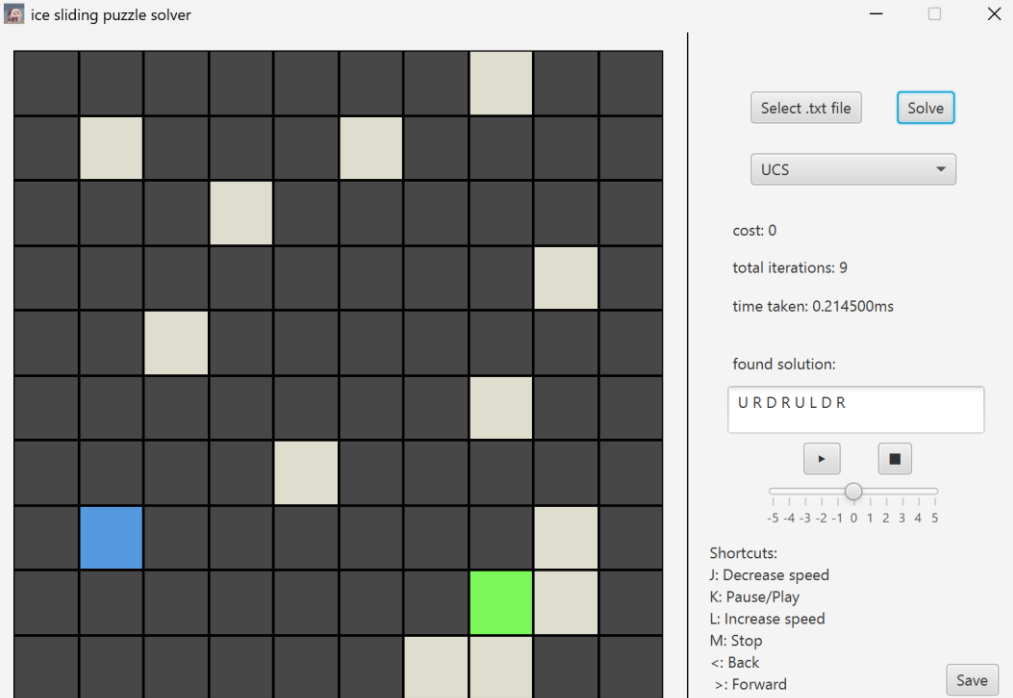
2. Test case 2

Input	<pre> 10 10 **X*X**X** *XO**X**** ***X***** *****X* *1X***** X***0**X** ****X***** *Z*****X* ***** *****X** 1 2 999 4 5 6 7 999 8 9 1 999 2 999 4 999 5 6 7 8 1 2 3 4 999 5 6 7 8 9 1 2 3 4 5 6 7 8 999 0 1 2 999 3 4 5 6 7 8 9 1 999 3 4 5 6 7 999 8 9 1 2 3 4 999 5 6 7 8 9 1 2 3 4 5 6 7 8 999 9 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 999 9 0 </pre> Metode: GBFS
Output	 The screenshot shows the 'ice sliding puzzle solver' interface. On the left is a 10x10 grid representing the puzzle state. It contains several blocks: a green block at (2,3), a blue block at (8,2), an orange block with the number '1' at (5,2), a red block with the number '0' at (6,5), and several yellow blocks. On the right, a sidebar provides solution details: 'cost: 2103', 'total iterations: 17', 'time taken: 0.195900ms', and the 'found solution: RULUDDLURU'. It also includes a speed slider and a list of shortcuts (J: Decrease speed, K: Pause/Play, L: Increase speed, M: Stop, <: Back, >: Forward) and a 'Save' button.
Analisis	Test case ini menunjukkan, player tidak dapat keluar dari map dan player hanya bisa berhenti jika menabrak tembok. Seperti test case sebelumnya, semua metode menghasilkan cost dan solusi yang sama.

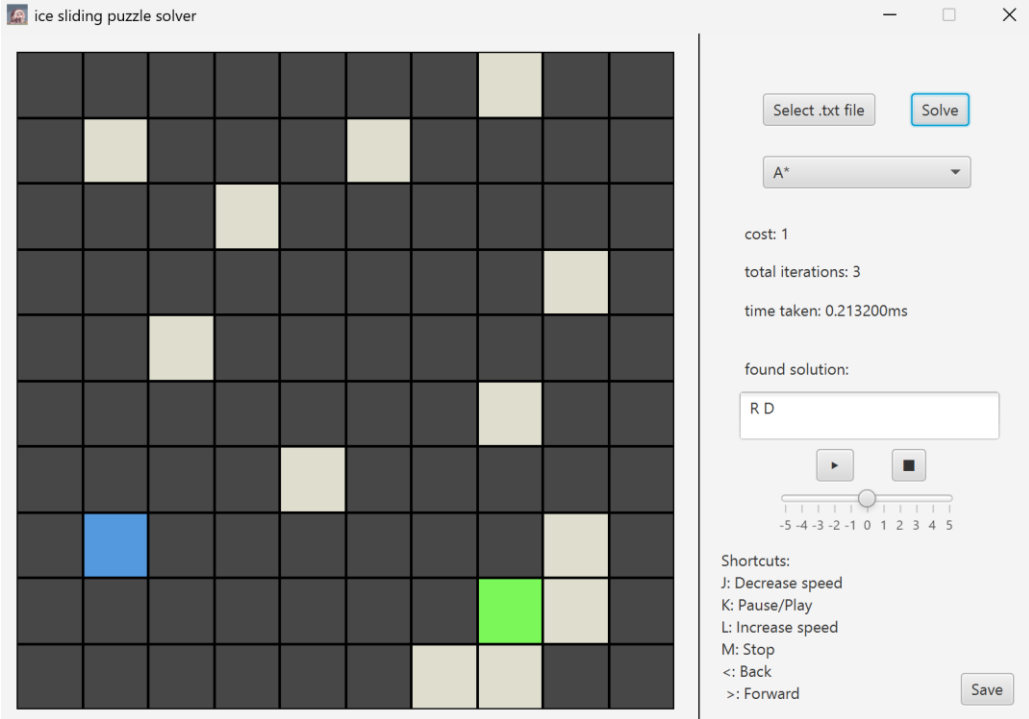
3. Test case 3

Input	<pre> 10 10 *****X** *X***X**** ***X***** *****X* **X***** *****X** ****X***** *Z*****X* ***** *****OX** 1 2 3 4 5 6 7 999 8 9 1 999 2 3 4 999 5 6 7 8 1 2 3 4 999 5 6 7 8 9 1 2 3 4 5 6 7 8 999 0 1 2 999 3 4 5 6 7 8 9 1 2 3 4 5 6 7 999 8 9 1 2 3 4 999 5 6 7 8 9 1 2 3 4 5 6 7 8 999 9 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 999 9 0 </pre> Metode: A*
Output	
Analisis	Test case ini memang tidak ada solusi, karena tidak ad acara untuk player tepat berhenti pada goal.

4. Test case 4

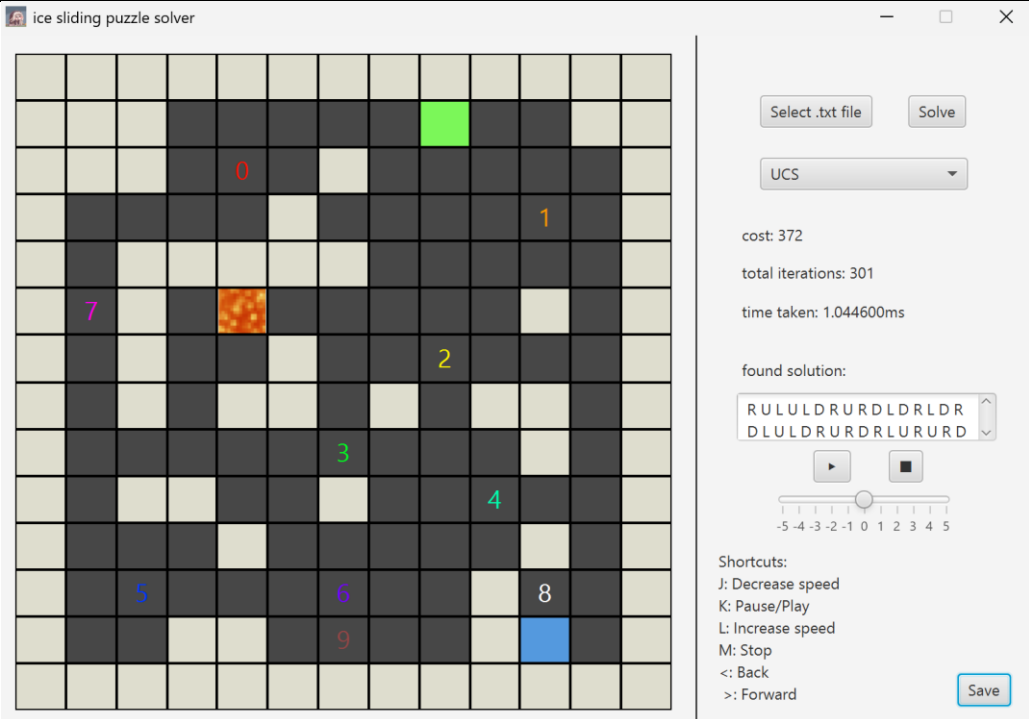
Input	<pre> 10 10 *****X** *X***X**** ***X***** *****X* **X***** *****X** ****X***** *Z*****X* *****OX* *****XX** 1 2 3 4 5 6 7 999 8 9 1 999 2 3 4 999 0 0 7 8 1 0 0 999 2 5 0 0 8 9 1 0 0 0 0 0 0 0 999 0 1 0 999 3 4 5 0 7 8 9 1 0 3 4 5 6 0 999 8 9 1 0 3 4 999 5 0 7 8 9 1 2 0 0 0 1 0 0 999 9 1 2 3 4 5 6 0 0 999 0 1 2 3 4 5 6 999 999 9 0 </pre> Metode: UCS
Output	 The screenshot shows the 'ice sliding puzzle solver' interface. The main grid is 10x10. A blue block is at row 7, column 2. A green block is at row 8, column 7. Yellow blocks are at (1,8), (2,2), (2,6), (3,4), (4,9), (5,3), (6,8), (7,9), (8,8), (9,7), and (9,8). The right sidebar shows the solution 'URDRULDR' and statistics: cost: 0, total iterations: 9, time taken: 0.214500ms. Shortcuts are listed: J: Decrease speed, K: Pause/Play, L: Increase speed, M: Stop, <: Back, >: Forward.
Analisis	Program menemukan solusi paling cost-efficient yakni 0 solusinya lebih panjang

5. Test case 5

Input	Sama seperti TC4 Metode: A*
Output	
Analisis	Hasil dari TC ini berbalik dari TC4 dimana solusinya lebih pendek tetapi cost akhirnya bukan yang paling optimal. Ini dikarenakan dalam kasus ini heuristic A* tidak admissible karena ada petak yang dilewati yang memiliki cost bernilai 0.

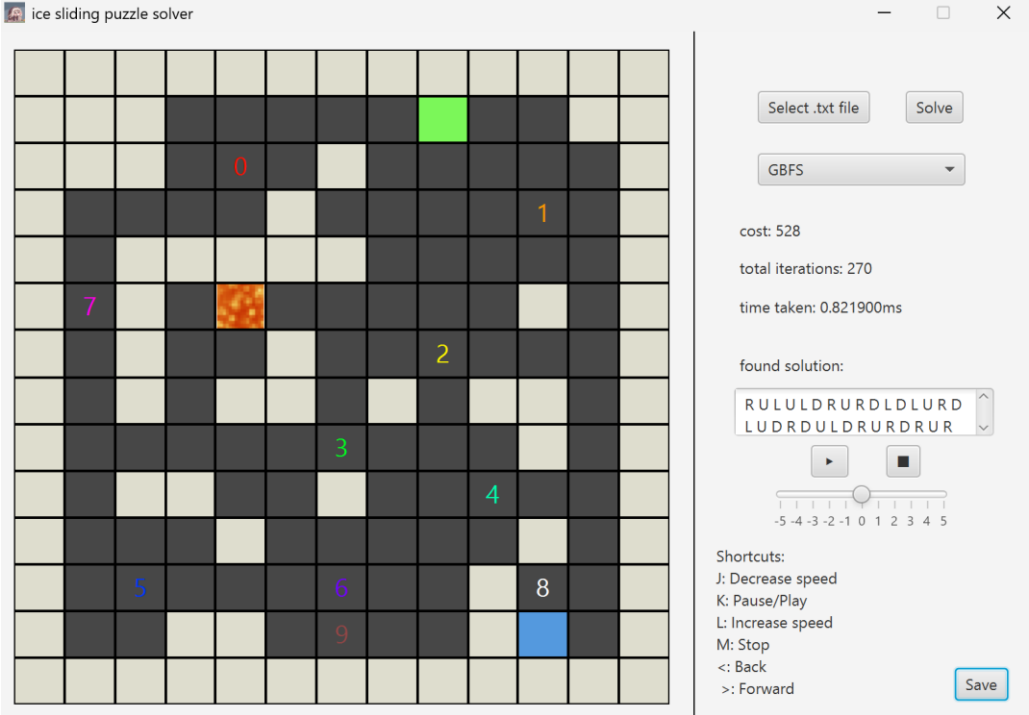
6. Test case 6

Input	<pre> 14 13 XXXXXXXXXXXXXXXXX XXX*****O**XX XXX*0*X*****X X****X*****1*X X*XXXXX*****X X7X*L*****X*X X*X**X**2***X X*X*XX*X*XX*X X*****3***X*X X*XX**X**4**X X***X*****X*X X*5***6**X8*X X**XX*9**XZ*X XXXXXXXXXXXXXXXXX 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 1 1 1 1 1 1 1 1 999 999 </pre>
-------	---

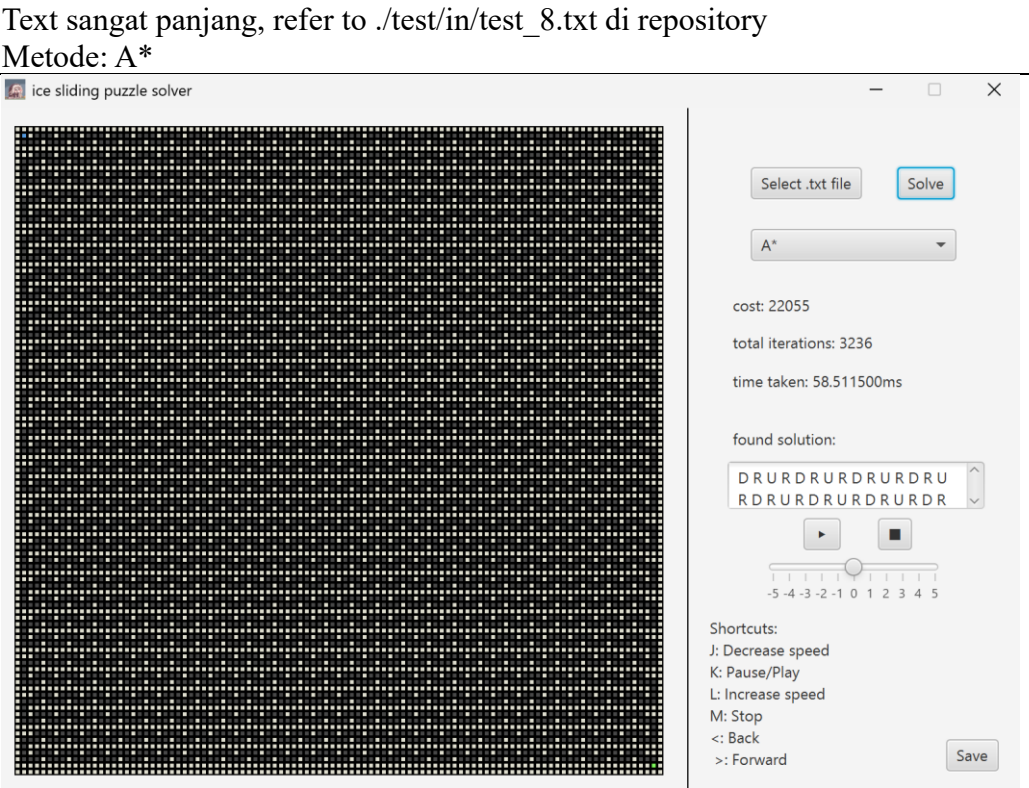
	<pre> 999 999 999 1 1 1 999 1 1 8 8 8 999 999 1 1 1 9 999 4 4 1 6 1 6 999 999 1 999 999 999 999 999 4 1 6 6 6 999 999 1 999 1 999 4 4 4 1 6 999 3 999 999 1 999 1 1 999 4 4 1 1 1 1 999 999 1 999 1 999 999 1 999 1 999 999 1 999 999 2 2 2 2 1 1 1 1 5 999 5 999 999 1 999 999 1 1 999 1 1 1 5 5 999 999 2 2 2 999 1 1 1 1 1 999 5 999 999 2 1 2 1 1 1 1 1 999 1 1 999 999 2 2 999 999 1 1 1 1 999 1 1 999 999 999 999 999 999 999 999 999 999 999 999 999 </pre>
Output	
Analisis	Again, UCS mendapat solusi yang efisien cost-wise

7. Test case 7

Input	Sama seperti TC6 Metode: GBFS
-------	---

Output	
Analisis	GBFS menghasilkan solusi yang tidak optimal, ini dikarenakan heuristic yang digunakan hanya mempertimbangkan jarak dari player ke goal.

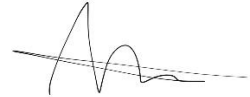
8. Test case 8

Input	Text sangat panjang, refer to ./test/in/test_8.txt di repository Metode: A*
Output	
Analisis	Uhhh, ini hanya untuk testing kecepatan program

IV. Lampiran

[Link Repository](#)

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



I Gusti Ngurah Alit Dharma Yudha

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		✓